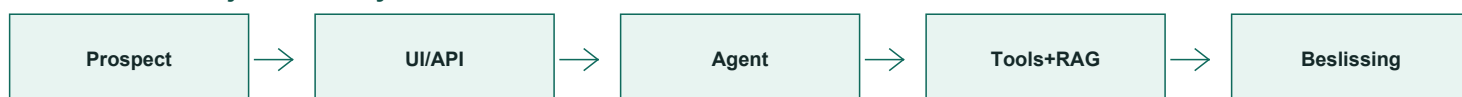


LexIntake

Uitgebreid Leerboek van Begin tot Eind

Alles wat je moet weten over de capstone-opdracht:
Agentic RAG intake screening voor advocatenkantoren
met Agno, PostgreSQL/pgvector, FastAPI, Streamlit, Docker en CI.

Schema: Grote lijn van het systeem



Schema: Lagenarchitectuur

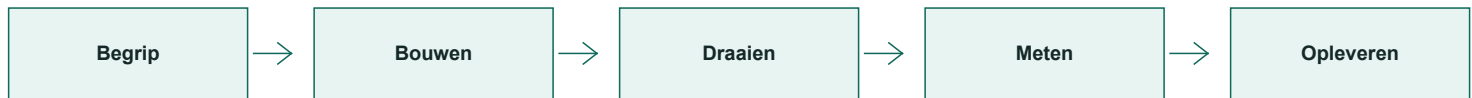
UI	Streamlit (frontend/) interview + quick analysis
API	FastAPI (backend/) REST endpoints
Agent	Plan / Retrieve / Tools / Decide / Self-check
Data	PostgreSQL: kb_docs (pgvector) + entities
KB/ETL	kb/ -> chunk -> embed -> load

*Doel: opdracht, architectuur, repo, start en uitleg leren.
GEEN juridisch advies — alleen educatief materiaal.*

Versie: LexIntake Capstone 2026
Repo: github.com/SteSel123/LexIntake

Inhoudsopgave

Schema: Hoe dit boek is opgebouwd



Schema: Leerpad in 5 delen

Deel A	Probleem, begrippen, eisen
Deel B	KB, ETL, DB, tools, agent
Deel C	Interview, scoring, guardrails
Deel D	Monitoring, eval, UI, CI
Deel E	Voorbeelden, cheatsheets, FAQ

1. Wat is de opdracht en waarom bestaat LexIntake?
2. Belangrijke begrippen (eenvoudig uitgelegd)
3. Capstone-eisen: wat moet je opleveren?
4. Architectuur: het grote plaatje
5. Repository-structuur map voor map
6. Bestandsgids: wat doet elk bestand?
7. Knowledge Base (kb/) bouwen
8. ETL-pipeline stap voor stap
9. Embeddings: OpenAI
10. PostgreSQL + pgvector (kb_docs)
11. Gestructureerde entiteiten in PostgreSQL
12. Docker Compose & FastAPI backend
13. Agno tools (SQL, conflict, value, route)
14. De Intake Agent (plan → retrieve → tools → decide)
15. Multi-turn interview met prospects
16. Lead scoring & beslissingen
17. Guardrails (juridische veiligheid)
18. Observability & Agno Monitoring
19. Evaluation harness
20. Streamlit UI & demo
21. Configuratie met .env

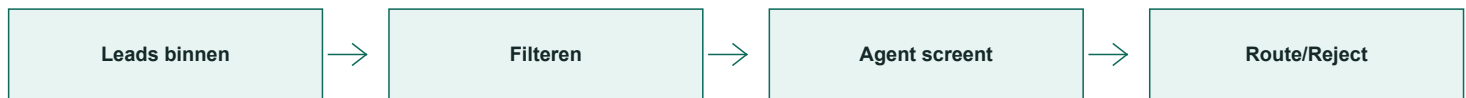
UI	Agent	RAG	Tools	Score	Guard
----	-------	-----	-------	-------	-------

- 22. Git-workflow, PR's en CI
- 23. Stap-voor-stap: systeem starten
- 24. Demo-video voorbereiden
- 25. Troubleshooting & veelgemaakte fouten
- 26. Oefeningen om te oefenen
- 27. Glossarium & checklist oplevering
- 28. Werkvoorbeelden (PI, SOL, conflict, uncertain)
- 29. Presentatiescript, dataflow, providers, ethiek
- 30. Unit tests & code-kwaliteit
- 31. Cheatsheets, zelftest en FAQ-verdieping

Let op: Lees dit boek sequentieel de eerste keer. Daarna kun je per hoofdstuk springen.

Hoofdstuk 1. Wat is de opdracht en waarom bestaat LexIntake?

Schema: Van probleem naar oplossing



Advocatenkantoren krijgen elke dag veel nieuwe leads: via de website, e-mail en telefoonformulieren. Een groot deel van de tijd van een paralegal gaat op aan filteren: verkeerde practice area, verkeerde staat (jurisdiction), verlopen verjaringstermijn (statute of limitations / SOL), belangenconflict, of een zaak die te weinig waard is.

LexIntake is géén chatbot die zomaar praat. Het is een agentic RAG-systeem: een agent die beslist welke kennis hij ophaalt, welke tools hij aanroept, en of hij genoeg bewijs heeft om een lead te scoren en door te sturen.

Wat moet de agent kunnen?

- Prospects interviewen (vragen stellen tot de feiten compleet genoeg zijn)
- Firm knowledge ophalen uit een knowledge base (RAG)
- Tools aanroepen (SOL-check, conflict, case value, routing)
- Leads scoren en een beslissing geven
- Elke beslissing uitleggen met KB-citaten
- Escaleren naar een mens bij onzekerheid

Wat is dit níet?

- Geen juridisch advies aan cliënten
- Geen volledige case-management / CRM-vervanging
- Geen vervanging van een advocaat

Onthoud de disclaimer die overal terugkomt: “This is not legal advice. Consult a licensed attorney.”

Hoofdstuk 2. Belangrijke begrippen (eenvoudig uitgelegd)

Schema: Kernbegrippen als lagen

RAG	Ophalen + genereren met bewijs
Agentic	Plant, kiest tools, beslist
Embedding	Tekst -> vector
Guardrail	Veiligheidsregels

RAG (Retrieval-Augmented Generation)

Eerst relevante tekstfragmenten ophalen uit een kennisbank, daarna pas antwoorden genereren. Zo ‘verzint’ het model minder en kan het citaten geven.

Agentic

Het systeem plant stappen, kiest tools, en beslist of er genoeg info is. Het volgt niet blind één vaste prompt.

Embedding

Een tekst als vector (lijst getallen). Vergelijkbare betekenissen liggen dicht bij elkaar in die ruimte.

Vector database

Database geoptimaliseerd om ‘meest gelijkende’ embeddings te zoeken (semantische search). Bij LexIntake: PostgreSQL met pgvector-extensie.

Chunk

Klein stukje tekst uit een groter document, handig om te indexeren en te citeren.

Tool calling

De agent roept een functie aan (bijv. `conflict_check`) met gestructureerde input/output.

SOL / Statute of Limitations

Verjaringstermijn: binnen hoeveel tijd moet een claim (globaal) nog geldig zijn.

Guardrail

Veiligheidsregel: disclaimer, geen juridisch advies, escalatie, citaties verplicht.

Deterministisch pad

Zelfde input → zelfde output, zonder externe LLM. Belangrijk voor CI/tests.

Provider-agnostic

Je kunt LLM/embeddings wisselen via config (.env), niet hardcoded aan één vendor.

Hoofdstuk 3. Capstone-eisen: wat moet je opleveren?

Schema: Deliverable-keten



De opdracht-PDF beschrijft leerdoelen, verplichte bouwstenen en deliverables. Hieronder de checklist in mensentaal.

Bouwstenen die er moeten zijn

- Synthetische knowledge base (practice areas, SOL, fees, cases, attorneys, clients, FAQs)
- ETL: extract → clean → dedupe → chunk → metadata → embeddings → load
- ETL-eigenschappen: re-runnable, idempotent, incremental
- Agent-loop: Plan → Retrieve → Tool Call → Decision → Self-check
- Tools: SOL, conflict, estimate_case_value, route_lead
- Lead output: qualified, lead_score, priority, decision, recommended_attorney
- Guardrails in elk antwoord
- Observability (tokens, cost, latency, tools, retrieval, escalaties, case value)
- Git: main/develop/feature/*, PRs, protected main, GitHub Actions (aanbevolen)
- Stack: Agno + LLM + embeddings + PostgreSQL/pgvector + FastAPI + Streamlit + Docker + pytest + monitoring
- Evaluation op ~20–30 labeled leads (8 dimensies)

Deliverables

- GitHub repository
- Working system
- Synthetic KB
- Evaluation Report
- Dashboard
- README
- Demo Video (dit moet jij nog opnemen)
- Design Report

Stretch goals (niet verplicht)

UI	Agent	RAG	Tools	Score	Guard
----	-------	-----	-------	-------	-------

- Corrective RAG
- Multi-agent team
- Human-in-the-loop console
- Persistent memory
- Uitgebreidere CI/CD (nightly full eval)

LexIntake dekt de MVP-eisen. Stretch is optioneel. De demo-video is de belangrijkste openstaande menselijke deliverable.

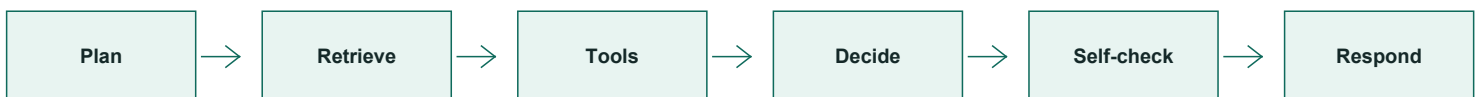
UI	Agent	RAG	Tools	Score	Guard
----	-------	-----	-------	-------	-------

Hoofdstuk 4. Architectuur: het grote plaatje

Schema: Architectuurlagen

UI	Streamlit (frontend/)
API	FastAPI (backend/)
Agent	IntakeAgent (Agno)
Tools	SOL / conflict / value / route
Data	PostgreSQL pgvector + entities
KB/ETL	kb/ -> embeddings

Schema: Agent-loop



Denk in lagen. Een prospect praat met de Streamlit UI (frontend/) of roept de FastAPI backend aan. Beide praten met de IntakeAgent. De agent haalt kennis uit PostgreSQL kb_docs (pgvector), roept tools aan die entiteiten/KB gebruiken, scoort de lead, checkt guardrails, en geeft een uitleg terug.

```

Prospect / paralegal
|
Streamlit UI (frontend/) — FastAPI (backend/)
|
IntakeAgent (agents/intake/)
plan -> retrieve -> tools -> decide -> self-check -> respond
|   |   |
Postgres Agno tools Lead scoring
kb_docs  SOL/conflict/value/route
(pgvector) |
^
clients / attorneys / past_cases
ETL (kb/ -> chunks -> embeddings)
Docker: postgres + init + api + ui
  
```

Live pad

- OpenAI embeddings + gpt-4.1 (echte semantiek + uitleg)
- OPENAI_API_KEY is verplicht voor ETL, UI, demo, evaluatie en CI

GitHub Actions gebruikt dezelfde live providers via de repository secret OPENAI_API_KEY.

Hoofdstuk 5. Repository-structuur map voor map

Schema: Repo-boom

```
LexIntake/
kb/          synthetische kennisbank (JSON/MD)
etl/         extract -> transform -> load
db/          Postgres models, pgvector, migrations
agents/      intake-agent + interview-agent + llm
tools/       Agno @tool functies
scoring/     deterministische lead scoring
backend/     FastAPI routes + services
frontend/    Streamlit UI + demo
monitoring/  metrics, dashboard, Agno traces
evaluation/  labeled leads + harness
scripts/     docker_init, ensure_env
docs/        reports + dit leerboek
docker-compose Postgres + init + API + UI
tests/       pytest unit tests
Makefile     make setup / test / ui / api / eval
.github/     CI smoke-test
```

LexIntake is opgebouwd als een monorepo: elke map heeft één duidelijke rol. Data stroomt van kb/ via etl/ naar PostgreSQL; runtime-logica zit in agents/, tools/ en scoring/; presentatie in frontend/ en backend/.

Schema: Volledige repo-boom

```
LexIntake/
├── kb/          # brondata (JSON + Markdown)
├── etl/         # extract → transform → load
│   ├── extract/ # leest kb-bestanden
│   ├── transform/ # clean, chunk, embed
│   └── load/     # schrijft naar Postgres
├── db/          # database-laag
│   ├── models.py # SQLAlchemy entities
│   ├── pgvector_store # kb_docs vector search
│   ├── engine.py # Postgres connectie
│   └── migrations/ # Alembic schema
├── agents/      # AI-agents
│   ├── intake/  # IntakeAgent (kern)
│   ├── interview/ # multi-turn gesprek
│   ├── llm.py   # provider-agnostic LLM
│   └── shared/   # Agno factory helpers
├── tools/       # Agno @tool functies
├── scoring/     # deterministische lead scoring
├── backend/     # FastAPI REST API
│   ├── api/routes/ # /health, /v1/intake, /v1/interview
│   └── services/   # business logic wrappers
├── frontend/    # Streamlit UI + demo
│   └── components/ # header, disclaimer, result viewer
├── monitoring/  # metrics, dashboard, Agno traces
├── evaluation/  # labeled leads + harness
├── scripts/     # docker_init, ensure_env
├── docs/        # reports + dit leerboek
├── docker-compose.yml # Postgres + init + API + UI
├── Dockerfile   # app image
├── tests/       # pytest unit tests (16+)
├── pyproject.toml # pytest pythonpath + package config
├── Makefile     # make setup / test / ui / api / eval
├── config.py    # .env configuratie
└── .github/workflows/ # CI: pytest + smoke-test
```

UI	Agent	RAG	Tools	Score	Guard
----	-------	-----	-------	-------	-------

kb/

Synthetische kennisbank (JSON/MD). Bron van waarheid voor intake-regels.

etl/

Pipeline: extract → clean → dedupe → chunk → metadata → embed → load.

db/

PostgreSQL: SQLAlchemy models, pgvector store, Alembic migrations, seed scripts.

agents/

IntakeAgent (plan/retrieve/tools/decide), InterviewAgent, LLM wiring.

tools/

Agno @tool functies: SOL, conflict, value, route.

scoring/

score_lead(), context builder, named constants (single source of truth).

tests/

Pytest: scoring, guardrails, fact_parse, conflict_check (geen API key).

backend/

FastAPI REST API (intake analyze, interview sessions, health).

frontend/

Streamlit UI (interview + quick analysis) + CLI demo scenarios.

monitoring/

JSONL logger, metrics, Streamlit dashboard, Agno traces.

evaluation/

leads.csv, eval_metrics, run_evaluation.py, report logs.

scripts/

docker_init.py (container bootstrap), ensure_env.py (.env helper).

docs/

Design report, evaluation report, demo script, dit leerboek.

Root-bestanden

- README.md — setup, architectuur, quick start
- requirements.txt — Python dependencies
- config.py — leest .env (DATABASE_URL, providers, models, keys)
- .env.example — template zonder geheimen
- docker-compose.yml — Postgres + init + API + UI containers
- Makefile — make setup, test, ui, api, demo, eval, dashboard
- pyproject.toml — pytest config + pip install -e . support
- Dockerfile — Python app image (PYTHONPATH=/app)

UI	Agent	RAG	Tools	Score	Guard
----	-------	-----	-------	-------	-------

Hoofdstuk 6. Bestandsgids: wat doet elk bestand?

Schema: Repo-boom

LexIntake/	
kb/	synthetische kennisbank (JSON/MD)
etl/	extract -> transform -> load
db/	Postgres models, pgvector, migrations
agents/	intake-agent + interview-agent + llm
tools/	Agno @tool functies
scoring/	deterministische lead scoring
backend/	FastAPI routes + services
frontend/	Streamlit UI + demo
monitoring/	metrics, dashboard, Agno traces
evaluation/	labeled leads + harness
scripts/	docker_init, ensure_env
docs/	reports + dit leerboek
docker-compose	Postgres + init + API + UI
tests/	pytest unit tests
Makefile	make setup / test / ui / api / eval
.github/	CI smoke-test

Deze tabel is je navigatiekaart. Als je een bug zoekt of iets moet uitleggen tijdens de demo, wijs je naar het juiste bestand.

Schema: Complete bestandsgids

UI	Agent	RAG	Tools	Score	Guard
----	-------	-----	-------	-------	-------

Bestand	Functie
config.py	Laadt .env: DATABASE_U
db/engine.py	PostgreSQL connectie +
db/models.py	SQLAlchemy: Client, At
db/pgvector_store.py	kb_docs upsert + vecto
db/schema.py	kb_docs DDL + embeddin
db/init_structured_db.	Maakt tabellen + seed
db/structured_db.py	CRUD helpers voor enti
db/migrations/	Alembic versies voor s
etl/pipeline.py	Hoofd-ETL: orchestrati
etl/extract/documents.	Leest alle kb/ bestand
etl/transform/clean.py	Normaliseert whitespac
etl/transform/deduplic	content_hash deduplica
etl/transform/chunk.py	Splitst tekst, genereer
etl/transform/metadata	Voegt practice_area, d
etl/transform/embeddin	OpenAI embedder via Ag
etl/load/vector_db.py	Upsert chunks naar kb_
agents/intake/agent.py	IntakeAgent: plan→retr
agents/intake/retrieve	pgvector search + meta
agents/intake/fact_par	Parse ruwe tekst naar
agents/intake/decide.p	Adapter: score_lead →
scoring/context.py	build_lead_score conte
scoring/constants.py	Named thresholds (SCOR
agents/intake/guardrai	Disclaimer, citaties,
agents/intake/models.p	Pydantic: IntakeFacts,
agents/intake/tools.py	Tool-aanroep wiring (a
agents/intake/prompts.	Systeem-instructies vo
agents/interview/agent	InterviewSession: mult
agents/llm.py	build_model(): OpenAI/
agents/shared/make_age	Agno Agent factory + t
tools/check_statute_of	SOL-check op sol_table
tools/conflict_check.p	Zoekt client match in
tools/estimate_case_va	Case value schatting v
tools/route_lead.py	Attorney routing op sp
tools/common.py	Gedeelde helpers (prac
scoring/lead_scoring.p	score_lead(): qualifie
backend/api/main.py	FastAPI app entrypoint
backend/api/routes/int	POST /v1/intake/analyz
backend/api/routes/int	Interview session endp

UI	Agent	RAG	Tools	Score	Guard
----	-------	-----	-------	-------	-------

backend/api/routes/hea

UI	Agent	RAG	Tools	Score	Guard
----	-------	-----	-------	-------	-------

GET /health

UI	Agent	RAG	Tools	Score	Guard
----	-------	-----	-------	-------	-------

backend/api/schemas.py

UI	Agent	RAG	Tools	Score	Guard
----	-------	-----	-------	-------	-------

Pydantic request/respo

UI	Agent	RAG	Tools	Score	Guard
----	-------	-----	-------	-------	-------

backend/api/session_st



UI	Agent	RAG	Tools	Score	Guard
----	-------	-----	-------	-------	-------

In-memory interview se

UI	Agent	RAG	Tools	Score	Guard
----	-------	-----	-------	-------	-------

backend/services/intak

UI	Agent	RAG	Tools	Score	Guard
----	-------	-----	-------	-------	-------

Gedeelde service: agen

UI	Agent	RAG	Tools	Score	Guard
----	-------	-----	-------	-------	-------

frontend/app.py

UI	Agent	RAG	Tools	Score	Guard
----	-------	-----	-------	-------	-------

Streamlit: interview t

UI	Agent	RAG	Tools	Score	Guard
----	-------	-----	-------	-------	-------

frontend/demo.py

UI	Agent	RAG	Tools	Score	Guard
----	-------	-----	-------	-------	-------

CLI demo: 4 scenario's

UI	Agent	RAG	Tools	Score	Guard
----	-------	-----	-------	-------	-------

frontend/runner.py

UI	Agent	RAG	Tools	Score	Guard
----	-------	-----	-------	-------	-------

Shared runner voor UI

UI	Agent	RAG	Tools	Score	Guard
----	-------	-----	-------	-------	-------

frontend/components/

UI	Agent	RAG	Tools	Score	Guard
----	-------	-----	-------	-------	-------

UI onderdelen: header,

UI	Agent	RAG	Tools	Score	Guard
----	-------	-----	-------	-------	-------

monitoring/logger.py

UI	Agent	RAG	Tools	Score	Guard
----	-------	-----	-------	-------	-------

.JSONL event logging +

UI	Agent	RAG	Tools	Score	Guard
----	-------	-----	-------	-------	-------

monitoring/metrics.py

UI	Agent	RAG	Tools	Score	Guard
----	-------	-----	-------	-------	-------

Tokens, latency, tool

UI	Agent	RAG	Tools	Score	Guard
----	-------	-----	-------	-------	-------

monitoring/dashboard.p

UI	Agent	RAG	Tools	Score	Guard
----	-------	-----	-------	-------	-------

Streamlit metrics + Ag

UI	Agent	RAG	Tools	Score	Guard
----	-------	-----	-------	-------	-------

monitoring/agno_tracin

UI	Agent	RAG	Tools	Score	Guard
----	-------	-----	-------	-------	-------

Agno OpenTelemetry tra

UI	Agent	RAG	Tools	Score	Guard
----	-------	-----	-------	-------	-------

evaluation/run_evaluat

UI	Agent	RAG	Tools	Score	Guard
----	-------	-----	-------	-------	-------

Draait agent op label

UI	Agent	RAG	Tools	Score	Guard
----	-------	-----	-------	-------	-------

evaluation/leads.csv

UI	Agent	RAG	Tools	Score	Guard
----	-------	-----	-------	-------	-------

~30 synthetische gelab

UI	Agent	RAG	Tools	Score	Guard
----	-------	-----	-------	-------	-------

evaluation/eval_metric

UI	Agent	RAG	Tools	Score	Guard
----	-------	-----	-------	-------	-------

Metric berekeningen pe

UI	Agent	RAG	Tools	Score	Guard
----	-------	-----	-------	-------	-------

scripts/docker_init.py

UI	Agent	RAG	Tools	Score	Guard
----	-------	-----	-------	-------	-------

Container init: init_s

UI	Agent	RAG	Tools	Score	Guard
----	-------	-----	-------	-------	-------

scripts/ensure_env.py

UI	Agent	RAG	Tools	Score	Guard
----	-------	-----	-------	-------	-------

Maakt .env aan vanuit

UI	Agent	RAG	Tools	Score	Guard
----	-------	-----	-------	-------	-------

tests/

UI	Agent	RAG	Tools	Score	Guard
----	-------	-----	-------	-------	-------

Pytest suite: lead_sco

UI	Agent	RAG	Tools	Score	Guard
----	-------	-----	-------	-------	-------

pyproject.toml

UI	Agent	RAG	Tools	Score	Guard
----	-------	-----	-------	-------	-------

pytest pythonpath + se

UI	Agent	RAG	Tools	Score	Guard
----	-------	-----	-------	-------	-------

[.github/workflows/ci.y](#)

UI	Agent	RAG	Tools	Score	Guard
----	-------	-----	-------	-------	-------

Cl: pytest, init DB, E



Dataflow tussen bestanden

Schema: Van brondata tot UI-antwoord



Let op: Imports draaien vanuit repo-root via `PYTHONPATH` (Makefile/Docker/CI zetten dit). Geen `sys.path`-hacks meer in library code. Optioneel: `pip install -e .`

Hoofdstuk 7. Knowledge Base (kb/) bouwen

Schema: KB-onderdelen

Document	Doel
practice_areas	Scope
acceptance	Qualify rules
sol_tables	Deadlines
past_cases	Valuation
attorneys	Routing
clients	Conflicts
faqs	Grounding

De KB is synthetisch maar realistisch. Ze bestaat zodat de agent iets heeft om op te ‘steunen’ (grounding) in plaats van regels te verzinnen.

Bestanden

- practice_areas.json — vaste lijst van 10 practice areas
- acceptance_criteria.json — wanneer neemt het kantoor een zaak aan?
- fee_structure.json — contingency/hourly/retainer per gebied
- sol_tables.json — SOL-regels per staat/zaaktype
- past_cases.json — vergelijkbare zaken voor valuation
- attorneys.json — specialisatie, ervaring, availability
- clients.json — bestaande cliënten voor conflict checks
- faqs.md — algemene groundingstekst

Practice areas (exact)

Personal Injury, Employment Law, Immigration, Family Law, Criminal Defense, Workers’ Compensation, Medical Malpractice, Product Liability, Civil Rights, Consumer Protection.

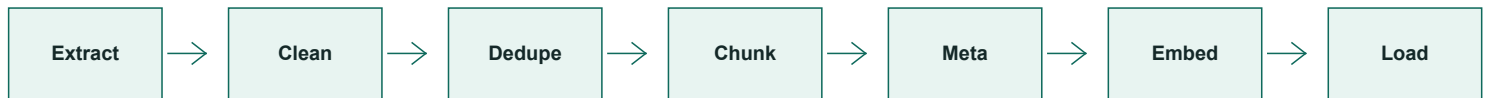
Waarom JSON + Markdown?

JSON is makkelijk te parsen voor tools (SOL/fees/cases). Markdown FAQs zijn goed te chunk’en voor retrieval. De ETL normaliseert alles tot documenten met metadata.

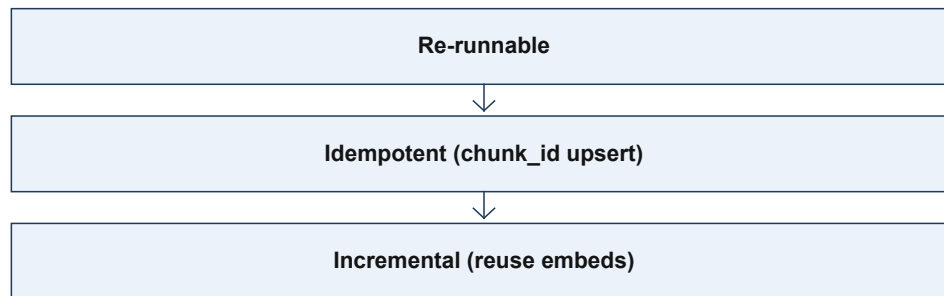
Let op: Wijzig je de KB, dan moet je embeddings opnieuw laden (python -m etl.pipeline) zodat kb_docs in Postgres synchroon blijft.

Hoofdstuk 8. ETL-pipeline stap voor stap

Schema: ETL-pijplijn



Schema: Verplichte ETL-eigenschappen



ETL = Extract, Transform, Load. Bij LexIntake is 'Transform' uitgebreid tot clean, dedupe, chunk, metadata, embeddings.

1. Extract

Lees alle KB-bestanden in als documenten.

2. Clean

Normaliseer whitespace/rommel, maak tekst consistent.

3. Deduplicate

Zelfde inhoud → één document via content_hash.

4. Chunk

Splits lange teksten in stukken met stabiele chunk_id.

5. Metadata

Voeg practice_area, jurisdictions, doc_type, timestamps toe.

6. Embeddings

Maak vectoren met OpenAI text-embedding-3-small.

7. Load

Upsert naar PostgreSQL kb_docs via pgvector_store.py.

Verplichte eigenschappen

- Re-runnable: je mag de pipeline steeds opnieuw draaien zonder chaos
- Idempotent: zelfde input convergeert naar dezelfde store-state (upsert op chunk_id)
- Incremental: ongewijzigde chunks kunnen embeddings hergebruiken

Commands

```
python -m db.init_structured_db
python -m etl.pipeline
```

Hoofdstuk 9. Embeddings: OpenAI

Schema: Embedding-modi

Pad	Model	Dims	API?
Live	text-emb-3-small	1536	Ja

Schema: Van tekst naar vector



Embeddings zetten tekst om naar getallen. LexIntake gebruikt OpenAI text-embedding-3-small via `get_embedder()`.

OpenAI text-embedding-3-small

- Dimensies: 1536
- Echte semantische gelijkenis (“rear-end collision” $\approx$ “car accident injury”)
- Vereist OPENAI_API_KEY
- Gebruikt via Agno OpenAIEmbedder adapter in `etl/transform/embeddings.py`

.env voorbeeld

```
LEXINTAKE_EMBEDDING_PROVIDER=openai  
LEXINTAKE_EMBEDDING_MODEL=text-embedding-3-small  
LEXINTAKE_EMBEDDING_DIMS=1536  
OPENAI_API_KEY=sk-...
```

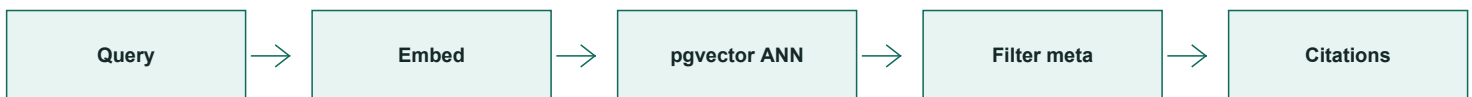
Let op: Wissel je van embedding-dimensie, dan moet je kb_docs opnieuw laden vanwege dimensie-mismatch. Dat is normaal.

Hoofdstuk 10. PostgreSQL + pgvector (kb_docs)

Schema: kb_docs schema (PostgreSQL)

chunk_id	Primary key voor upsert
text	Chunk inhoud
embedding	pgvector (1536 dims, HNSW)
payload	JSONB type-specifieke velden
jurisdictions	text[] met GIN-filter

Schema: Retrieval-pad



LexIntake gebruikt één PostgreSQL-database voor alles. Vector-chunks staan in tabel kb_docs met de pgvector-extensie. Zoeken gebeurt via HNSW-index op cosine similarity, met metadata filters.

Schema (kernvelden)

- chunk_id — primaire sleutel voor idempotente upsert
- text — chunk tekst
- embedding — pgvector kolom (1536 dims)
- practice_area — slug voor filtering
- jurisdictions — text[] array met GIN-index
- doc_type — bv. sol_rules, acceptance, faq
- payload — JSONB met type-specifieke velden

Belangrijke bestanden

- db/pgvector_store.py — upsert + vector_search
- db/schema.py — DDL + embedding dimensies
- agents/intake/retrieve.py — retrieval met filters
- tools/common.py — vector_search wrapper

Retrieval in de agent

De agent embedt de query via OpenAI, zoekt top_k via pgvector (typisch 5–10) en filtert op practice_area / jurisdiction / doc_type. Resultaten worden citaties.

```

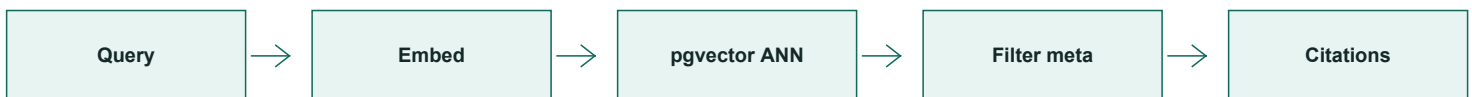
hits = vector_search(
    query, top_k=8,
    practice_area='Personal Injury',
    jurisdiction='CA', doc_type='sol_rules')
  
```

Hoofdstuk 11. Gestructureerde entiteiten in PostgreSQL

Schema: kb_docs schema (PostgreSQL)

chunk_id	Primary key voor upsert
text	Chunk inhoud
embedding	pgvector (1536 dims, HNSW)
payload	JSONB type-specifieke velden
jurisdictions	text[] met GIN-filter

Schema: Retrieval-pad



Entities met relaties en exacte lookups zitten in dezelfde PostgreSQL-database als kb_docs (niet meer in aparte SQLite).

Tabellen

- clients — voor conflict_check (naam/opposing party)
- attorneys — voor route_lead (specialisatie, availability, jurisdictions)
- past_cases — voor estimate_case_value (comps), FK naar clients + attorneys

Belangrijke bestanden

- db/models.py — SQLAlchemy ORM (Client, Attorney, PastCase)
- db/structured_db.py — query helpers voor tools
- db/init_structured_db.py — schema + seed vanuit kb/*.json (python -m db.init_structured_db)
- db/migrations/ — Alembic (001 schema, 002 pgvector)

Seed

python -m db.init_structured_db maakt tabellen aan en vult vanuit kb/*.json. Conflict-demo's werken omdat seeded clients (zoals Elena Vasquez) bestaan.

Hoofdstuk 12. Docker Compose & FastAPI backend

Schema: Docker Compose services

postgres	pgvector DB container
init	schema seed + ETL (eenmalig)
api	FastAPI op :8000
ui	Streamlit op :8501

LexIntake draait standaard als Docker-stack via docker-compose.yml: postgres, init (eenmalig), api (FastAPI :8000) en ui (Streamlit :8501).

Docker services

- postgres — pgvector/pgvector:pg16, poort 5432
- init — scripts/docker_init.py: init_structured_db + etl.pipeline
- api — backend/api/main.py via uvicorn
- ui — frontend/app.py via streamlit

FastAPI endpoints

- GET /health — health check
- GET / — service discovery (docs + route hints)
- POST /v1/intake/analyze — quick analysis (één casebeschrijving)
- POST /v1/interview/sessions — start interview sessie
- POST /v1/interview/sessions/{id}/turns — antwoord sturen
- DELETE /v1/interview/sessions/{id} — sessie opruimen

Backend API-tests staan in tests/test_backend_api.py (FastAPI TestClient, pipeline gemockt zodat CI geen live LLM nodig heeft voor HTTP-coverage).

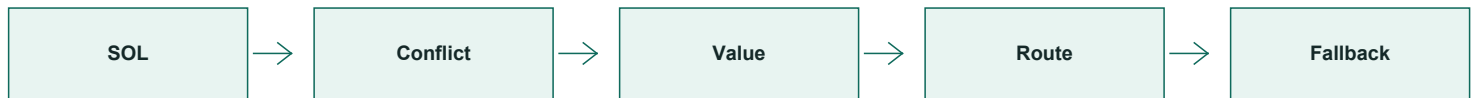
Makefile

```
make setup      # volledige Docker stack
make setup-local # host Python + Postgres container
make ui / api   # Streamlit / FastAPI lokaal
make demo / eval # demo + evaluation
```

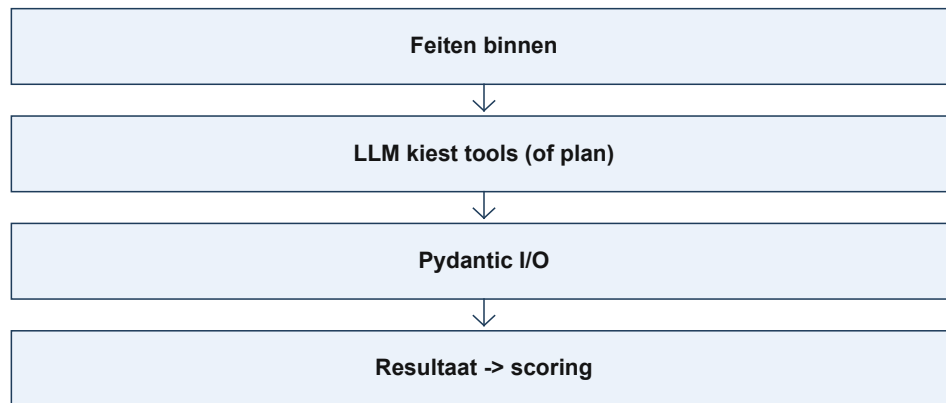
Let op: DATABASE_URL is verplicht: postgresql://lexintake:lexintake@localhost:5432/lexintake

Hoofdstuk 13. Agno tools (SOL, conflict, value, route)

Schema: Toolketen



Schema: Tool-aanroepstructuur



Tools zijn gewone Python-functies met Pydantic input/output, gewrapt als Agno `@tool` zodat de LLM ze kan aanroepen.

check_statute_of_limitations

Input: `jurisdiction`, `case_type`, `incident_date`. Output: `valid`, `expires_in` (dagen), `explanation`. Leest `sol_tables.json`.

conflict_check

Input: `name`, `opposing_party`. Zoekt in Postgres `clients` tabel. Output: `conflict` bool + details.

estimate_case_value

Input: `case_type`, `severity`, `damages`. Blend van `comps` + `stated damages`. Output: `estimate` + `range` + `explanation`.

route_lead

Input: `practice_area`, `priority`. Kiest attorney op specialisatie/load. Output: `attorney_name` + `rationale`.

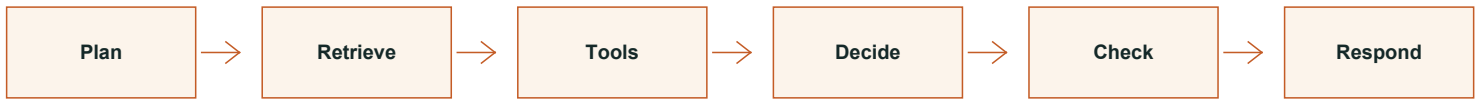
Agentic vs deterministisch

Met LLM: `Agent.run(..., tool_choice='auto')` laat het model tools kiezen. Zonder LLM (CI): de plan-fase bepaalt tools en roept endpoints direct aan. Ontbrekende toolresultaten worden deterministisch aangevuld voor betrouwbaarheid.

UI	Agent	RAG	Tools	Score	Guard
----	-------	-----	-------	-------	-------

Hoofdstuk 14. De Intake Agent (plan → retrieve → tools → decide)

Schema: Agentische loop



Bij onvoldoende info: terug naar vragen / retrieve / escalate

Bestand: agents/intake/agent.py. Klasse IntakeAgent erft van Agno Agent.

Plan

Welke velden missen? Welke tools? Welke retrieval-query? Escaleren?

Retrieve

pgvector semantic search + filters → citaties.

Tools

Agentic of deterministisch tool-aanroepen.

Decide

Lead score, viability, routing, next steps, confidence.

Self-check

Disclaimer? Citaten? Verboden taal? Onzekerheid?

Respond

Gebruikersbericht (LLM of template) + gestructureerde response.

Waarom deze loop?

De capstone eist expliciete agentic workflow. Door fasen te scheiden kun je monitoren (latency per stap), testen, en uitleggen wat er gebeurt — belangrijker dan één ondoorzichtige ‘black box’ prompt.

Hoofdstuk 15. Multi-turn interview met prospects

Schema: Interview-flow



Schema: Verplichte intake-velden

Veld	Voorbeeld
name	Alex Rivera
practice_area	Personal Injury
jurisdiction	CA
incident_date	2025-06-01
opposing_party	City Transit
damages	45000

De PDF vraagt om 'Interview prospective clients'. Daarom bestaat er een InterviewSession (agents/interview/agent.py), een Streamlit-tab, en een statische web-UI op Vercel (web/) die dezelfde FastAPI-routes gebruikt.

Hoe werkt het?

- Agent start met welkom + disclaimer + eerste vraag
- Gebruiker antwoordt in chat (web-UI toont een date picker bij datumvragen)
- Sessie vult IntakeFacts (heuristiek + optioneel LLM-extractie)
- Zolang verplichte velden missen: nieuwe vragen
- Ongeldige antwoorden (geen echte US-staat, geen parsebare datum) worden niet opgeslagen → agent blijft doorvragen
- Als genoeg info (of 'screen now' mét practice area + geldige staat): run_intake() screening
- Eindigt met samenvatting + guardrails + citaties

Verplichte velden

name, practice_area, jurisdiction, incident_date, opposing_party, damages

Datum & jurisdiction

- incident_date: ISO YYYY-MM-DD, 'X months/years ago', 06/15/2024, June 15 2024 — genormaliseerd via infer_incident_date(); anders None
- jurisdiction: alleen echte US-state codes/namen (CA, California, ...); geen cleaned[:2]-gok meer (bugfix: 'Personal Injury' werd 'PE')
- SOL-tool draait alleen bij valide ISO-datum; anders skip + log

Quick analysis blijft bestaan

Voor demoscenario's en evaluatie is single-pass (plak tekst → analyse) handig. Interview is de conversatie-ervaring; quick analysis is de snelle pipeline-demo.

Hoofdstuk 16. Lead scoring & beslissingen

Schema: Unified scoring-pad



Schema: Beslissingsruimte

Decision	Betekenis
SCHEDULE_CONSULT	Plan intake
REVIEW	Menselijke check
REJECT	Niet aannemen

LexIntake heeft één scoring-engine: `score_lead()` in `scoring/lead_scoring.py`. Zowel de `IntakeAgent` (via `agents/intake/decide.py`) als de `API/UI` (via `backend/services/intake_service.py`) gebruiken dezelfde context-builder in `scoring/context.py`. Geen dubbele score-logica meer.

Scoring-keten (single source of truth)

Schema: Unified scoring



Belangrijke bestanden

- `scoring/lead_scoring.py` — `score_lead()`: qualified, score, decision, explanation
- `scoring/context.py` — `build_lead_score_context()` + acceptance criteria uit KB
- `scoring/constants.py` — drempels (70/40), boosts, penalties (named constants)
- `agents/intake/decide.py` — mapt `LeadScoreOutput` → `DecisionResult` voor agent-loop
- `backend/services/intake_service.py` — zelfde `score_lead` voor JSON API/UI

Deterministisch: zelfde context → zelfde `LeadScoreOutput`. Testbaar via `pytest` zonder `OPENAI_API_KEY`.

Outputvelden

```

{
  "qualified": true,
  "lead_score": 78,
  "priority": "High",
  "decision": "SCHEDULE_CONSULT",
  "recommended_attorney": "Jordan Hale",
  "explanation": "..."
}
```

Beslissingen

- `SCHEDULE_CONSULT` — voldoende fit, geen harde blockers
- `REVIEW` — onzeker / incomplete / menselijke check nodig
- `REJECT` — harde blockers (bv. conflict of duidelijk verlopen SOL)

UI	Agent	RAG	Tools	Score	Guard
----	-------	-----	-------	-------	-------

Harde regels (intuïtie)

- Conflict gevonden → meestal REJECT + escalatie
- SOL invalid → sterk negatief / reject pad
- Weinige acceptance matches → lagere score / review
- Hoge estimated value + goede fit → hogere priority

Hoofdstuk 17. Guardrails (juridische veiligheid)

Schema: Guardrail-stapel



Elke user-facing response moet aan deze eisen voldoen:

- Legal disclaimer aanwezig
- Nooit voorschrijvend juridisch advies (“you should sue” etc.)
- Escalate bij onzekerheid
- Citeer retrieved evidence (chunk_id, practice_area, doc_type)
- Weiger unsupported / verzonnen statutes of attorney profiles

Waar wordt dit afgedwongen?

- Agent instructions
- self_check fase
- respond() plakt disclaimer/escalatie/citaties indien nodig
- Evaluation check_guardrails()
- UI disclaimer componenten

Let op: Guardrails zijn een hard beoordelingspunt. Liever te vaak escaleren dan zelfverzekerd fout juridisch advies geven.

UI	Agent	RAG	Tools	Score	Guard
----	-------	-----	-------	-------	-------

Hoofdstuk 18. Observability & Agno Monitoring

Schema: Observability-stack

Custom JSONL	latency/tools/scores
Metrics API	session/daily summary
Streamlit dash	Grafieken
Agno tracing	traces.db spans

De opdracht vraagt monitoring van sessies. LexIntake heeft twee lagen:

1) Custom metrics (monitoring/)

- logger.py — JSONL events, PII-sanitization
- metrics.py — tokens, cost, latency per stap, tool calls, retrieval hit rate, lead scores, escalaties, case values
- dashboard.py — Streamlit grafieken

2) Agno native tracing (OpenTelemetry)

- monitoring/agno_tracing.py roept setup_tracing() aan
- Traces landen in monitoring/traces.db
- Per-call tokens/cost komen uit Agno run.metrics (geen custom estimate_cost / len(prompt)//4 meer)
- IntakeAgent sommeert die Agno-metrics over de sessie voor IntakeResponse
- Zichtbaar in dashboard sectie ‘Agno Monitoring’
- Uit te zetten met LEXINTAKE_AGNO_TRACING=0

```
python -m streamlit run monitoring/dashboard.py
```

Hoofdstuk 19. Evaluation harness

Schema: Evaluatieketen



Schema: Eval-dimensies (sample)

Dimensie	Meet
Retrieval	Hit rate
Grounding	Citations
Qualify	Label match
Guardrails	Compliance
Cost/Lat	Efficiency

evaluation/leads.csv bevat ~30 labeled synthetische leads. run_evaluation.py draait de agent en meet kwaliteit.

Acht dimensies

- Retrieval quality
- Grounding
- Qualification accuracy
- Case valuation
- Abstention behaviour
- Guardrails
- Cost & latency
- Provider comparison

Commands

```
# Live OpenAI (key vereist)
python evaluation/run_evaluation.py --providers openai:gpt-4.1 --limit 5
```

Rapport: docs/EVALUATION_REPORT.md. Perfecte qualificatie-accuracy is niet het doel; transparantie over grounding/guardrails wel.

Hoofdstuk 20. Streamlit UI & demo

Schema: UI Interview-pad



Schema: UI Quick-analysis-pad



App (frontend/)

```
make ui
# of: python -m streamlit run frontend/app.py
# Docker: http://localhost:8501
```

- frontend/app.py — hoofd-UI met interview + quick analysis tabs
- frontend/components/ — header, disclaimer, result_viewer, footer
- frontend/demo.py — CLI demo met 4 scenario's
- frontend/runner.py — gedeelde pipeline runner
- web/ — statische Vercel-UI (date picker bij incident_date; praat met Render API)
- Tab Interview — multi-turn gesprek via agents/interview/
- Tab Quick analysis — plak case description → run_intake
- Sidebar met demoscenarió's
- Result viewer: score, decision, citaties, guardrails, tool JSON

Deploy-URLs (typisch)

- API (Render): <https://lexintake-api.onrender.com>
- Frontend preview (Vercel develop): <https://lexintake-git-develop-app-assist.vercel.app>
- Postgres (Render): aparte managed DB lexintake-db via DATABASE_URL

API alternatief (backend/)

Dezelfde intake-logica via FastAPI op <http://localhost:8000/docs> (of de Render-URL). Handig voor integraties of als je UI en backend wilt scheiden. HTTP-coverage: `tests/test_backend_api.py`.

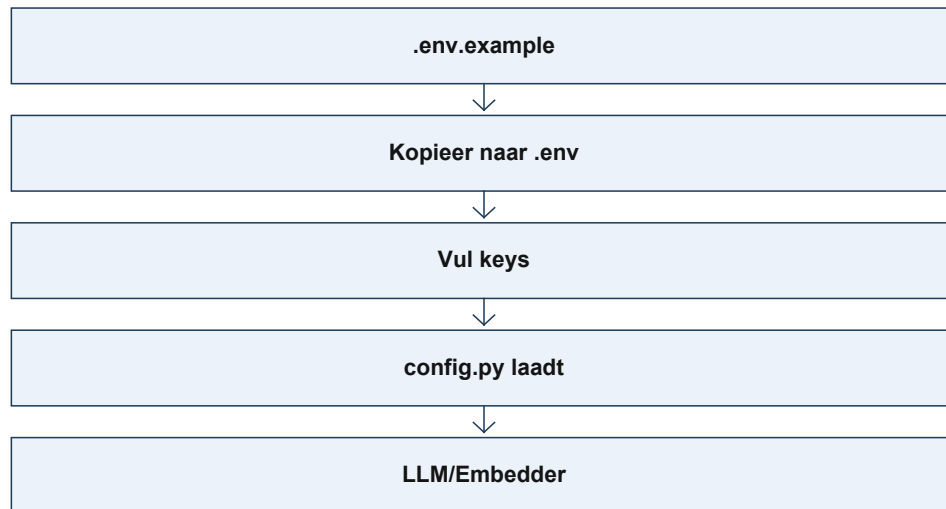
CLI demo (4 scenario's)

```
make demo # of: python frontend/demo.py
```

Scenario's: valid PI → SCHEDULE_CONSULT; expired SOL → REJECT; conflict ACME → REJECT+escalate; uncertain immigration → REVIEW+escalate.

Hoofdstuk 21. Configuratie met .env

Schema: Config-flow



Secrets horen in .env (gitignored). Gebruik .env.example als template.

```
DATABASE_URL=postgresql://lexintake:lexintake@localhost:5432/lexintake
OPENAI_API_KEY=
LEXINTAKE_EMBEDDING_PROVIDER=openai
LEXINTAKE_EMBEDDING_MODEL=text-embedding-3-small
LEXINTAKE_EMBEDDING_DIMS=1536
LEXINTAKE_LLM_PROVIDER=openai
LEXINTAKE_LLM_MODEL=gpt-4.1
LEXINTAKE_AGNO_TRACING=1
ANTHROPIC_API_KEY=
GROQ_API_KEY=
```

Veiligheid

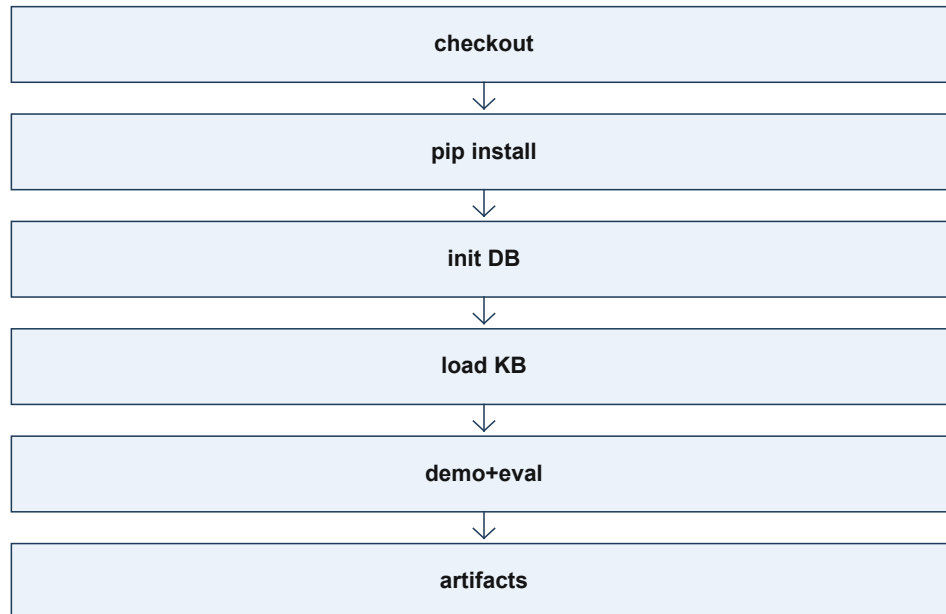
- Nooit keys in chat of commits plakken
- Gelekte key meteen intrekken en roteren
- config.py laadt dotenv; libraries lezen OPENAI_API_KEY uit env

Hoofdstuk 22. Git-workflow, PR's en CI

Schema: Git + CI flow



Schema: CI smoke-test stappen



Branches

- main — protected production
- develop — integratie
- feature/* — features

CI (.github/workflows/ci.yml)

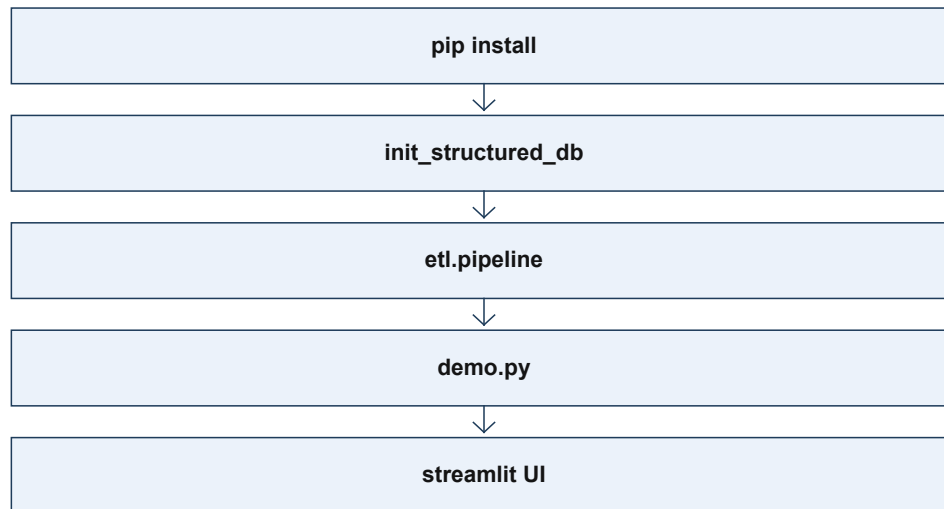
Draait op pull_request naar main, job smoke-test, met live OpenAI:

- Python 3.11 + pip install (PYTHONPATH=.)
- pytest tests/ — unit tests (scoring, guardrails, tools, geen API key)
- python -m db.init_structured_db
- etl.pipeline (OpenAI embeddings)
- frontend/demo.py (4 scenario's)
- evaluation --limit 5 (openai:gpt-4.1)
- upload logs als artifacts

main vereist status check smoke-test. Daardoor blokkeert een falende CI een merge.

Hoofdstuk 23. Stap-voor-stap: systeem starten

Schema: Startvolgorde



Optie A — Docker (aanbevolen)

```

copy .env.example .env
# Zet OPENAI_API_KEY in .env
make setup
# UI: http://localhost:8501
# API: http://localhost:8000/docs
  
```

Optie B — Lokaal Python + Postgres container

```

copy .env.example .env
# Zet OPENAI_API_KEY + DATABASE_URL in .env
make setup-local
make demo
make ui
make api # optioneel
  
```

Vereisten

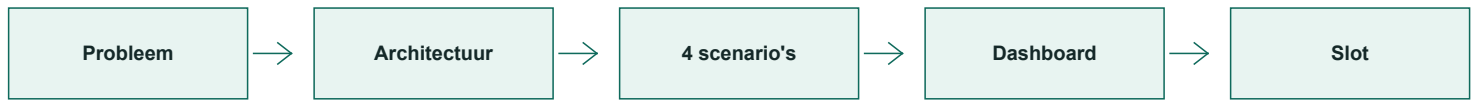
OPENAI_API_KEY én DATABASE_URL zijn verplicht voor ETL/demo/eval. Unit tests (make test) draaien zonder API key. PYTHONPATH wordt gezet door make/Docker/CI; lokaal: `pip install -e .` of `$env:PYTHONPATH='.'`

Handige checks

- Demo 4/4 PASS
- Interview tab stelt vragen en eindigt met screening
- Dashboard laadt metrics / Agno traces
- Geen .env in git status

Hoofdstuk 24. Demo-video voorbereiden

Schema: Video-storyboard



Gebruik docs/DEMO.md. Suggestie voor 3–5 minuten:

- 0:00 probleem + “geen chatbot, wel agentic RAG”
- 0:20 repo/architectuur in 30 seconden
- 0:50 live UI: valid PI scenario
- 1:40 SOL expired → REJECT
- 2:10 conflict → escalate
- 2:40 uncertain → REVIEW
- 3:10 monitoring dashboard
- 3:40 evaluation highlight (grounding/guardrails)
- 4:10 afsluiting + disclaimer

Toon altijd de disclaimer on-screen. Upload naar Loom/YouTube unlisted en zet de link in de README.

Hoofdstuk 25. Troubleshooting & veelgemaakte fouten

Schema: Debug-matrix

Symptoom	Check
Import error	venv + requirements
Key missing	.env OPENAI_API_KEY
Dim mismatch	herlaad KB
CI rood	secret + demo + eval

ModuleNotFoundError llm / embeddings

Zorg dat je vanuit repo-root draait en requirements geïnstalleerd zijn.

OPENAI_API_KEY missing

Zet OPENAI_API_KEY in .env (verplicht).

DATABASE_URL missing

Start Postgres: make db-up of docker compose up -d postgres.

Dimensie mismatch kb_docs

Herlaad met python -m etl.pipeline als embedding-dimensies wijzigen.

Conflict-demo faalt

init_structured_db opnieuw draaien zodat Elena Vasquez geseeded is.

CI rood

Lokaal nabootsen: .env key + demo + eval --limit 5. CI vereist secret OPENAI_API_KEY.

IntakeFacts validation error

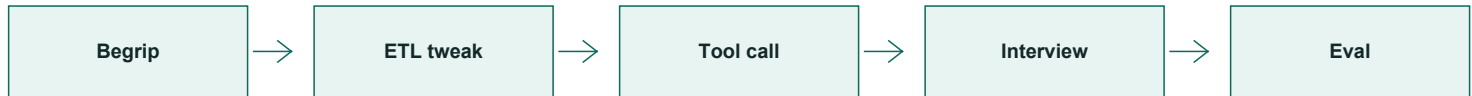
Dual imports; recente fix herbindt naar één model via model_dump.

Key gelekt in chat

Intrekken, nieuwe maken, nooit opnieuw plakken in issues/PR's.

Hoofdstuk 26. Oefeningen om te oefenen

Schema: Oefenparcours



Oefening A — Begrip

Leg in 10 zinnen uit aan een klasgenoot wat LexIntake doet zonder code te tonen. Gebruik de woorden: lead, RAG, tool, guardrail, escalatie.

Oefening B — ETL

Voeg één FAQ-regel toe in kb/faqs.md, run `python -m etl.pipeline`, en toon dat retrieval de nieuwe chunk kan vinden.

Oefening C — Tools

Roep `check_statute_of_limitations` aan voor CA/Personal Injury met een oude en een recente `incident_date`. Vergelijk `valid/expires_in`.

Oefening D — Interview

Doe een interview in de UI waarbij je expres jurisdiction weglaat of 'I don't know' typt. Controleer dat de agent doorvraagt (geen nep-state zoals PE). Test ook een datum als 'June 15, 2024' of de date picker.

Oefening E — Eval

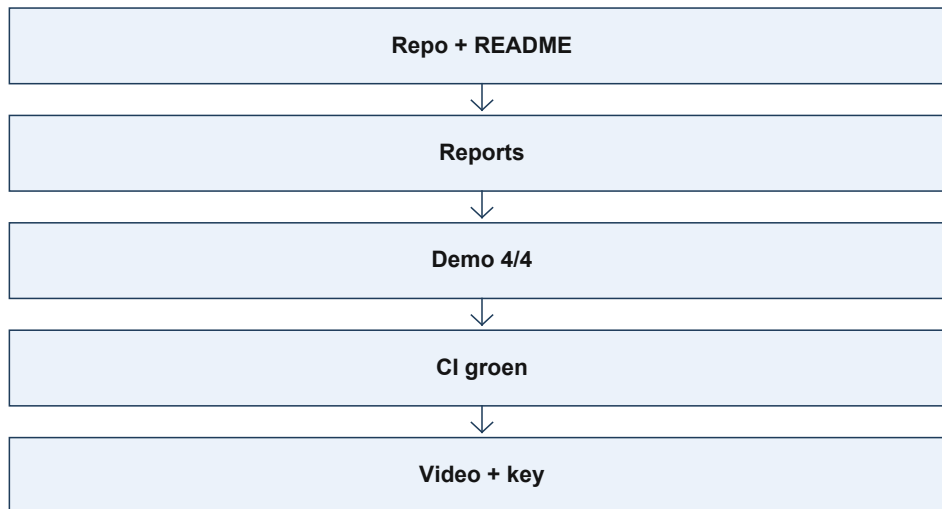
Draai `evaluation --limit 5` met OpenAI en noteer `grounding + guardrail compliance`.

Oefening F — Uitlegscore

Neem één lead en schrijf met de hand waarom de decision `SCHEDULE/REVIEW/REJECT` is, op basis van `SOL/conflict/value/acceptance`.

Hoofdstuk 27. Glossarium & checklist oplevering

Schema: Oplevervolgorde



Mini-glossarium

- Agno — agent framework
- pgvector — PostgreSQL vector extensie voor semantic search
- FastAPI — Python REST API framework (backend/)
- Docker Compose — container orkestratie (postgres + api + ui)
- chunk_id — stabiele id van een tekstbrok
- upsert — insert-or-update
- abstention — systeem onthoudt zich / escaleert
- grounding — antwoorden steunen op retrieved evidence
- smoke-test — snelle CI-check dat het systeem niet kapot is
- pytest — unit test framework (tests/ map, make test)
- PYTHONPATH — repo-root op import path (Makefile/Docker/CI)

Opleverchecklist

- Repo publiek/deelbaar met instructor
- README klopt (setup + disclaimer)
- Design Report + Evaluation Report aanwezig
- Dashboard draait
- Demo 4/4 groen
- make test / pytest groen (16+ unit tests)
- CI groen op main PRs (pytest + smoke)
- main protected
- OPENAI key lokaal gezet (niet gecommitt)
- Demo-video opgenomen + link toegevoegd
- Je kunt de architectuur hardop uitleggen

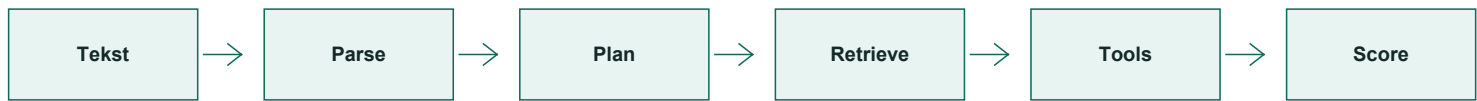
Laatste advies

UI	Agent	RAG	Tools	Score	Guard
----	-------	-----	-------	-------	-------

Beoordelaars kijken naar: werkt het end-to-end, is het agentic/RAG echt, zijn guardrails zichtbaar, is Git professioneel, en kun jij het uitleggen. Perfecte ML-metrics zijn minder belangrijk dan een helder, veilig intake-verhaal.

Hoofdstuk 28. Werkvoorbeeld end-to-end: Valid Personal Injury

Schema: Runtime end-to-end



Dit hoofdstuk volgt één lead van ruwe tekst tot beslissing. Lees het alsof je meekijkt in de runtime.

Input

Rear-end collision in CA, clear liability, \$45k damages, incident 6 months ago.

Stap 1 — Fact parsing

parse_case_description herkent 'rear-end/collision' als Personal Injury, staat CA, damages 45000, incident_date ≈ vandaag minus 6*30 dagen, severity medium/high afhankelijk van keywords, naam Demo Prospect.

Stap 2 — Plan

Omdat jurisdiction, case_type en incident_date bekend zijn, plant de agent check_statute_of_limitations. Met name+party komt conflict_check. Met damages komt estimate_case_value. Met practice_area komt route_lead. Retrieval staat aan.

Stap 3 — Retrieve

De query mix practice area, CA, narrative en woorden als acceptance/settlement. pgvector search geeft chunks terug (acceptance_criteria, sol_rules, past_case, faq). Die chunks worden citaties.

Stap 4 — Tools

- SOL CA/PI: meestal valid=True, expires_in positief (2-jaar regel)
- Conflict: geen match → conflict=False
- Value: blend comps + 45k → schatting in de orde van tonnen kan voorkomen door sparse comps; interpretatie: richtinggevend, niet exact
- Route: attorney met PI-specialisatie, bv. Jordan Hale

Stap 5 — Scoring

score_lead ziet geen conflict, SOL ok, practice match → qualified True, priority High, decision SCHEDULE_CONSULT.

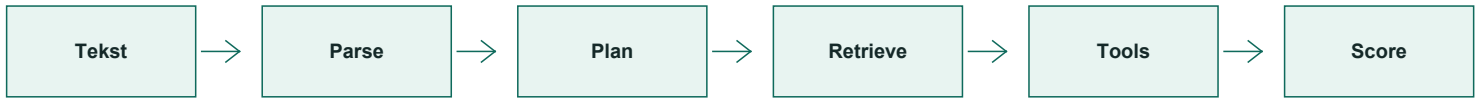
Stap 6 — Guardrails

Response bevat disclaimer, citaties, geen 'you should sue'. Escalatie niet nodig.

UI	Agent	RAG	Tools	Score	Guard
----	-------	-----	-------	-------	-------

Hoofdstuk 29. Werkvoorbeeld: verlopen SOL

Schema: Runtime end-to-end



Slip-and-fall in NV, incident 4 years ago, moderate injuries.

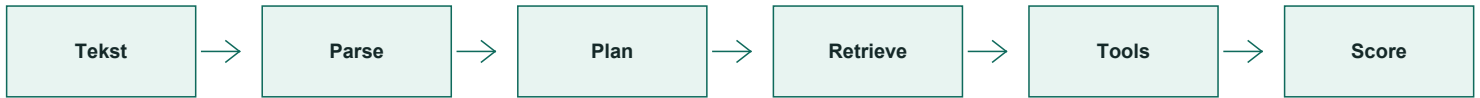
Parser: Personal Injury, NV, incident_date ver in het verleden. SOL-tool: valid=False, expires_in negatief. Scoring duwt naar REJECT. Uitleg noemt statute/expiration. Dit toont dat tools niet cosmetisch zijn: ze veranderen de beslissing.

In de demo checkt verify_scenario dat de explanation SOL-taal bevat. Zo bewijs je grounding + tool-effect in één scenario.



Hoofdstuk 30. Werkvoorbeeld: conflict of interest

Schema: Runtime end-to-end



Employment discrimination claim in CA, opposing party is ACME Corp.

De parser map't dit bewust naar client Elena Vasquez (seeded conflict case) wanneer employment+ACME voorkomt. `conflict_check` vindt een match in Postgres clients. Beslissing: REJECT of minstens escalatie. Belangrijk lespunt: structured DB is essentieel; vector search alleen is niet genoeg voor exacte identity checks.

UI	Agent	RAG	Tools	Score	Guard
----	-------	-----	-------	-------	-------

Hoofdstuk 31. Werkvoorbeeld: onzekere immigratiezaak

Schema: Runtime end-to-end

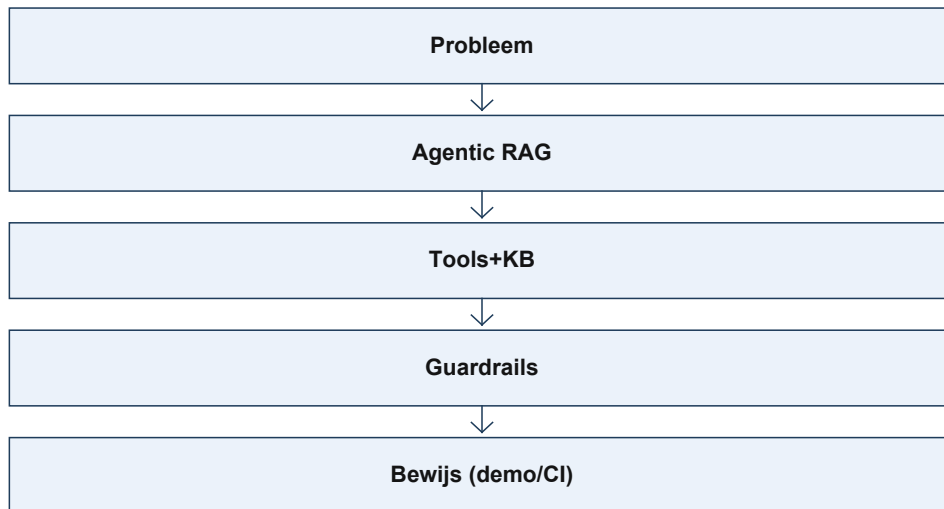


Immigration matter with unclear facts, missing dates, missing jurisdiction.

Veel velden ontbreken. Runner markeert uncertain. Zelfs als scoring niet hard reject, forceert UI/runner REVIEW + escalatie. Dit is abstention behaviour: liever menselijke review dan overconfident automatisering.

Hoofdstuk 32. Hoe leg je LexIntake uit in 3 minuten?

Schema: Pitch-structuur



Gebruik dit spreekscript:

- Probleem: kantoren verdrinken in leads; paralegals filteren handmatig.
- Oplossing: agentic RAG screener — niet chatbot.
- Kennis: synthetische KB + ETL naar PostgreSQL (pgvector + entities).
- Agent: plant, retrieve't, roept tools, scoort, self-checkt.
- Veiligheid: disclaimer, citaties, escalatie, geen legal advice.
- Bewijs: demo 4 scenario's, evaluation metrics, CI smoke, GitHub flow.

Sluit af met: 'The system recommends intake actions for staff; it does not advise clients. This is not legal advice.'

UI	Agent	RAG	Tools	Score	Guard
----	-------	-----	-------	-------	-------

Hoofdstuk 33. Dataflow in detail (van disk tot antwoord)

Schema: Dataflow 30.000 ft



1) Bestanden in kb/ liggen op disk. 2) extract_all leest ze. 3) clean normaliseert. 4) dedupe berekent content_hash. 5) chunk snijdt tekst en maakt chunk_id. 6) metadata hangt practice_area/doc_type/jurisdictions. 7) embedder maakt vectoren. 8) pgvector_store upsert naar kb_docs. 9) Bij runtime embedt de agent de query. 10) vector_search rangschikt via pgvector. 11) Tools lezen JSON/Postgres. 12) score_lead beslist. 13) respond bouwt tekst + JSON voor UI.

Als je dit rijtje kunt opschrijven zonder te spieken, begrijp je het project.

UI	Agent	RAG	Tools	Score	Guard
----	-------	-----	-------	-------	-------

Hoofdstuk 34. Provider-agnostic ontwerp

Schema: Provider-scheiding

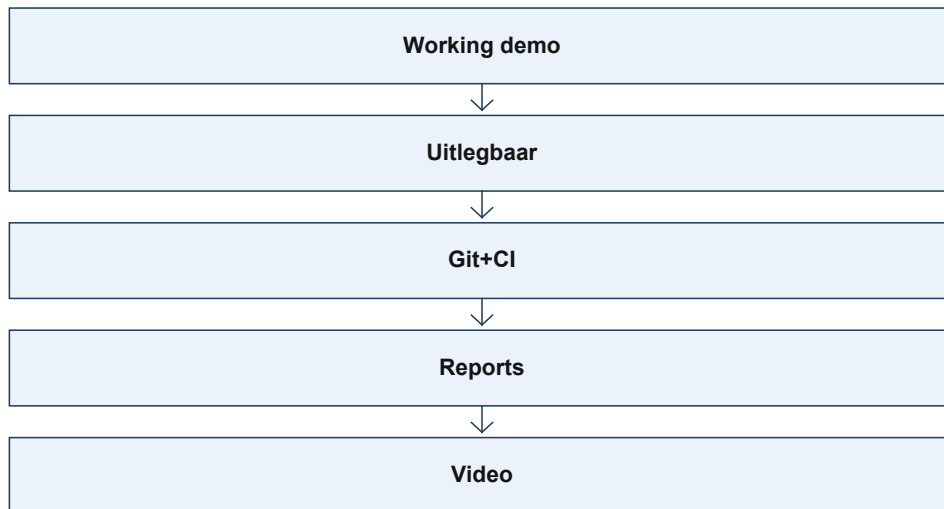
Laag	Keuze
LLM	openai/anthropic/groq
Embed	openai
Logic	onveranderd

config.py + agents/llm.py + etl/transform/embeddings.py scheiden ‘welke vendor’ van ‘welke business logic’. Daardoor kun je OpenAI, Anthropic of Groq kiezen voor LLM, en OpenAI voor embeddings, zonder tools/scoring te herschrijven.

CI gebruikt dezelfde live OpenAI-paden via de repository secret OPENAI_API_KEY.

Hoofdstuk 35. Wat is ‘goed genoeg’ voor grading?

Schema: Grading-prioriteiten



- Working demo die 4 scenario's correct toont
- Je kunt architectuur en guardrails uitleggen
- Git history + PR + CI zichtbaar
- Reports aanwezig en eerlijk over zwakke metrics (valuation)
- Video die het verhaal toont

Zwakke case-value accuracy is oké als je uitlegt waarom (sparse comps) en hoe je het zou verbeteren. Hallucinaties zonder citaties zijn niet oké.

Hoofdstuk 36. Veiligheids- en ethiekles

Schema: Veiligheidslagen

No advice	Geen directives aan prospect
Escalate	Twijfel -> mens
Cite	KB evidence
Secrets	Keys niet in git

Legal AI heeft asymmetrisch risico: een fout advies kan schade veroorzaken. Daarom: geen directives aan prospects, wel interne screening-samenvattingen; escalatie bij twijfel; logging zonder ruwe PII waar mogelijk; keys niet in git.

Als iemand vraagt ‘Moet ik aanklagen?’ is het juiste systeemgedrag: weigeren juridisch advies te geven en doorverwijzen naar een licensed attorney.

Hoofdstuk 37. Uitbreidingen als je tijd over hebt

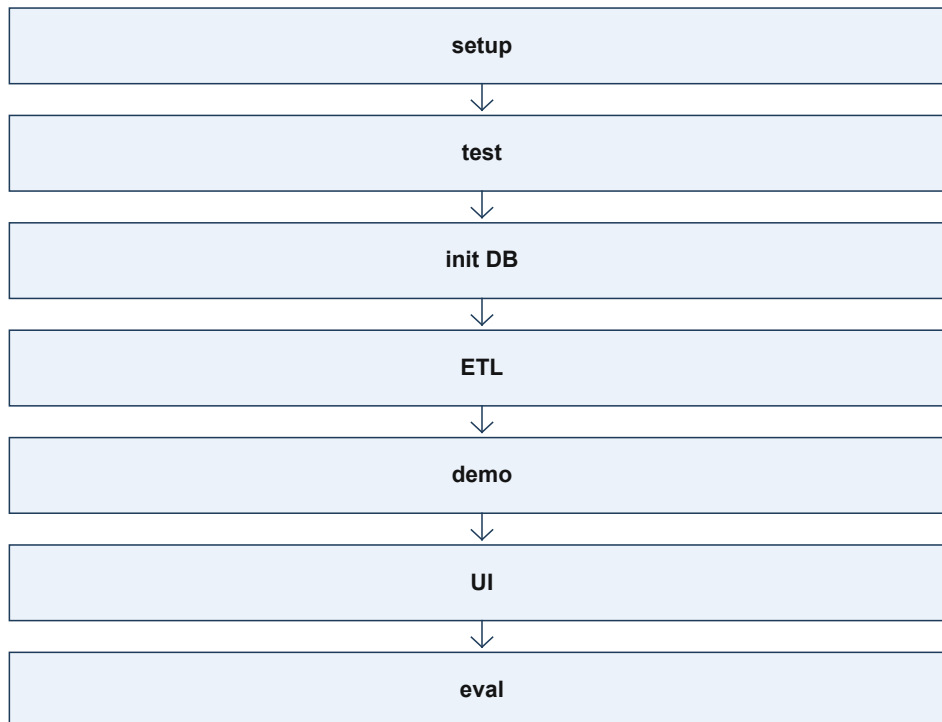
Schema: Stretch roadmap



- Corrective RAG: als citaties zwak zijn, herbreek query
- Multi-agent: aparte agents voor SOL vs conflict vs narration
- HITL console: paralegal keurt SCHEDULE_CONSULT goed/af
- Persistent memory: herhaalde callers herkennen
- Rijkere SOL-tabellen met numerieke years i.p.v. alleen free text
- Meer past_cases per practice area voor betere valuation

Hoofdstuk 38. Unit tests & code-kwaliteit

Schema: Command-volgorde



LexIntake heeft een pytest-suite in tests/ voor deterministische modules. Deze tests draaien snel en zonder OPENAI_API_KEY — ideaal vóór elke commit.

Wat wordt getest?

- scoring/lead_scoring.py — determinisme, SOL reject, conflict reject, score bands
- agents/intake/decide.py — decide() == score_lead() (unified scoring)
- scoring/context.py — acceptance criteria uit facts (geen placeholders)
- agents/intake/guardrails.py — disclaimer + citation checks
- agents/intake/fact_parse.py — PI case parsing heuristics
- tools/conflict_check.py — gemockte Postgres query

Commands

```

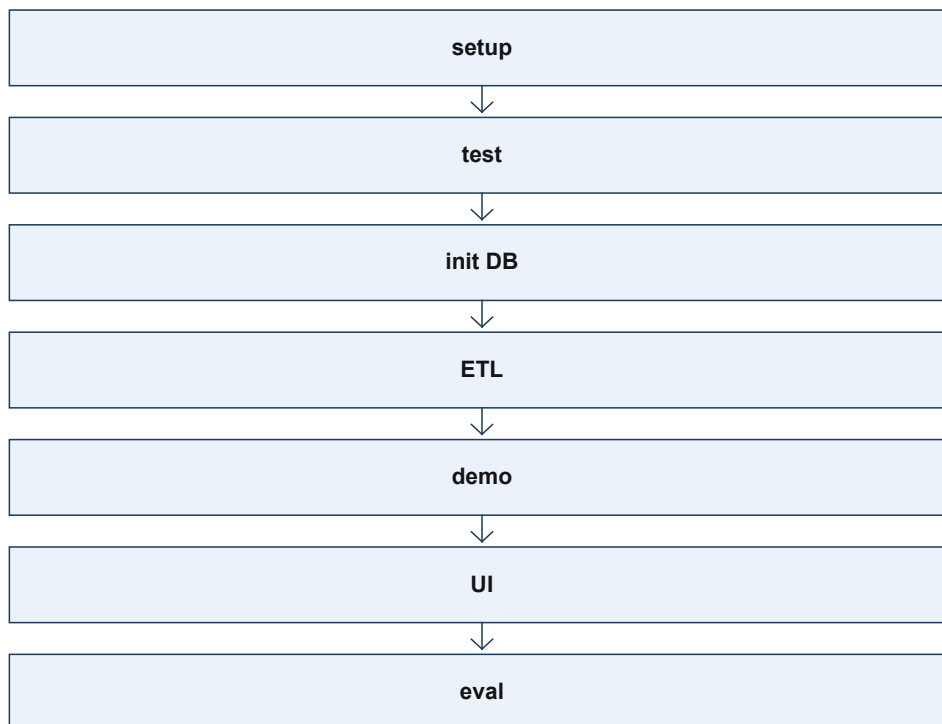
make test          # pytest tests/ -q
pytest tests/test_lead_scoring.py -v
pip install -e .   # optioneel: editable install
  
```

Code-kwaliteit principes in dit project

- Single source of truth — één score_lead(), één LEGAL_DISCLAIMER, één slugify
- Package imports — from tools.common import (geen sys.path hacks)
- Named constants — scoring/constants.py i.p.v. magic numbers
- Service layer — backend/services/intake_service.py deelt logica UI/API
- Deterministic core — business rules testbaar zonder LLM

Hoofdstuk 39. Cheatsheet: commands die je moet kennen

Schema: Command-volgorde



```
make setup      # Docker: Postgres + init + API + UI
make setup-local # Host Python + Postgres container
make test       # pytest (geen API key nodig)
make demo / ui / api # Demo, Streamlit, FastAPI
make eval / dashboard # Evaluation + monitoring
make down       # Stop containers
python docs/generate_leerboek_pdf.py # Genereer dit leerboek
```


Hoofdstuk 40. Cheatsheet: bestanden die je moet kunnen aanwijzen

Schema: Bestanden om aan te wijzen

```
agents/intake/agent.py    <- agent loop
db/pgvector_store.py     <- vector search
tools/*.py               <- SQL/conflict/value/route
backend/api/main.py      <- FastAPI entry
frontend/app.py          <- Streamlit UI
docker-compose.yml       <- stack
etl/pipeline.py          <- ETL
.github/workflows/ci.yml  <- CI
```

- Opdrachtbegrip -> docs/DESIGN_REPORT.md + dit leerboek
- KB brondata -> kb/*.json + faqs.md
- ETL -> etl/pipeline.py + etl/extract|transform|load/
- Vector store -> db/pgvector_store.py + db/schema.py
- Entities -> db/models.py + db/structured_db.py + db/migrations/
- Tools -> tools/*.py + tools/common.py
- Agent loop -> agents/intake/agent.py + retrieve.py + decide.py
- Interview -> agents/interview/agent.py
- LLM wiring -> agents/llm.py + agents/shared/make_agent.py
- Scoring -> scoring/lead_scoring.py + scoring/context.py + scoring/constants.py
- Tests -> tests/ (pytest, make test)
- UI -> frontend/app.py + frontend/components/
- API -> backend/api/main.py + backend/services/
- Docker -> docker-compose.yml + scripts/docker_init.py
- Eval -> evaluation/run_evaluation.py + leads.csv
- CI -> .github/workflows/ci.yml
- Config -> config.py + .env + Makefile

Hoofdstuk 41. Zelftest (antwoord ondersteboven in je hoofd)

Schema: Zelftest-kaart

Vraag	Focus
RAG vs LLM?	Grounding
Postgres entities waar	Exact match
5 fasen?	Loop
CI keys?	OPENAI_API_KEY secret

- Wat is het verschil tussen RAG en 'gewoon een LLM vragen'?
- Waarom Postgres entities naast pgvector kb_docs?
- Noem de 5 agent-fasen in volgorde.
- Welke beslissingen kan score_lead geven?
- Hoe dwingt CI live embeddings/LLM af?
- Waar zit de enige scoring-engine?
- Welke tests draaien zonder API key?
- Wat moet er altijd in een user-facing antwoord staan?
- Welk scenario toont conflict detection?
- Waar landen Agno traces?

Als je deze acht vragen soepel beantwoordt, ben je klaar om te presenteren.

UI	Agent	RAG	Tools	Score	Guard
----	-------	-----	-------	-------	-------

Einde van het LexIntake Leerboek

Je hebt nu een route van probleemstelling naar implementatie, runtime, evaluatie en oplevering. Gebruik de repo als lab: wijzig iets kleins, draai demo/eval, en observeer het effect.

This is not legal advice. Consult a licensed attorney for legal guidance.

Dit document is uitsluitend bedoeld om de capstone-opdracht te leren.